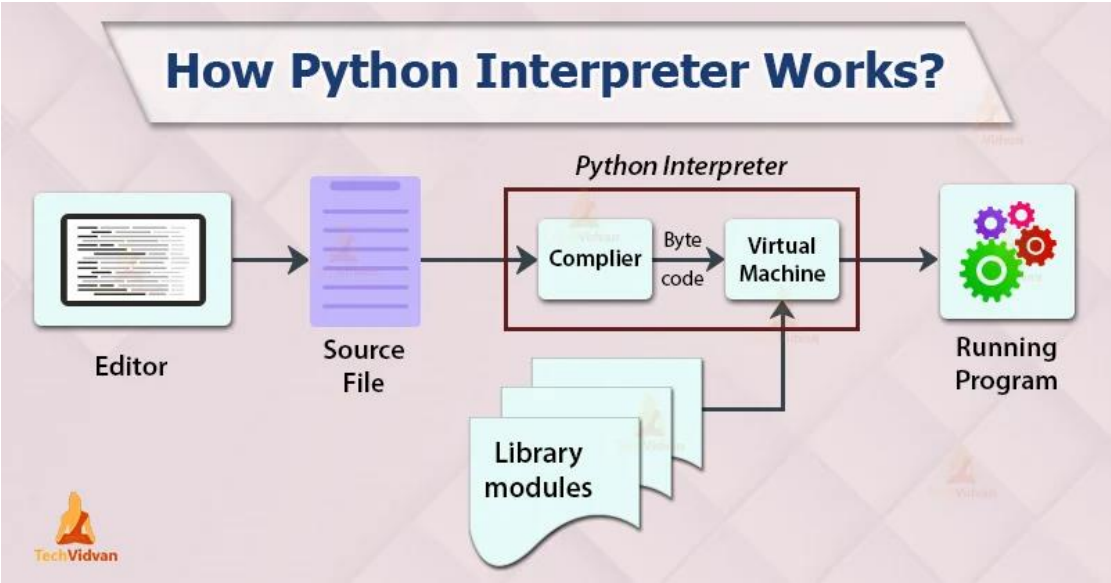
COM161 (75777)- Problem Solving for Computing. 2025-26.

COM161 - Problem Solving for Computing

Dr. Marwan M Radwan

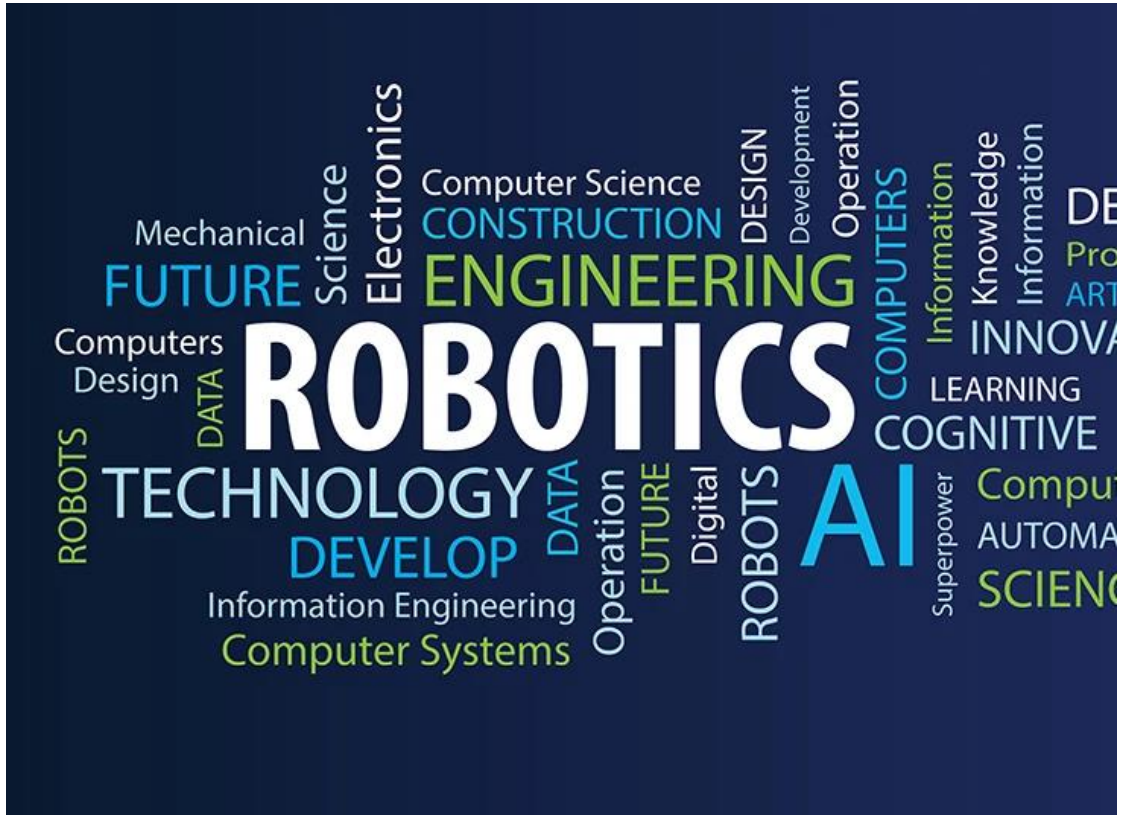
MRADWAN@QA.COM

Lecture, Week-4



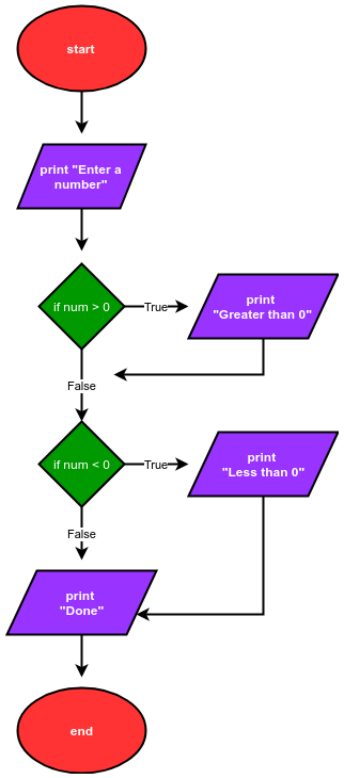
Warming Up … What Is Python?

- High-level programming language
  - Interpreted (Line-by-Line vs Compilation to Machine Code)
  - Dynamically typed
  - Multi-paradigm (procedural, object-oriented, functional)



Why Python?

- Readable syntax
  - Large ecosystem
  - Used in data science, AI, web, automation
  - Beginner-friendly but industry-grade



How Python Executes Code?

- Top-to-bottom execution
  - Indentation defines blocks
  - Errors stop execution

What we learned so far?

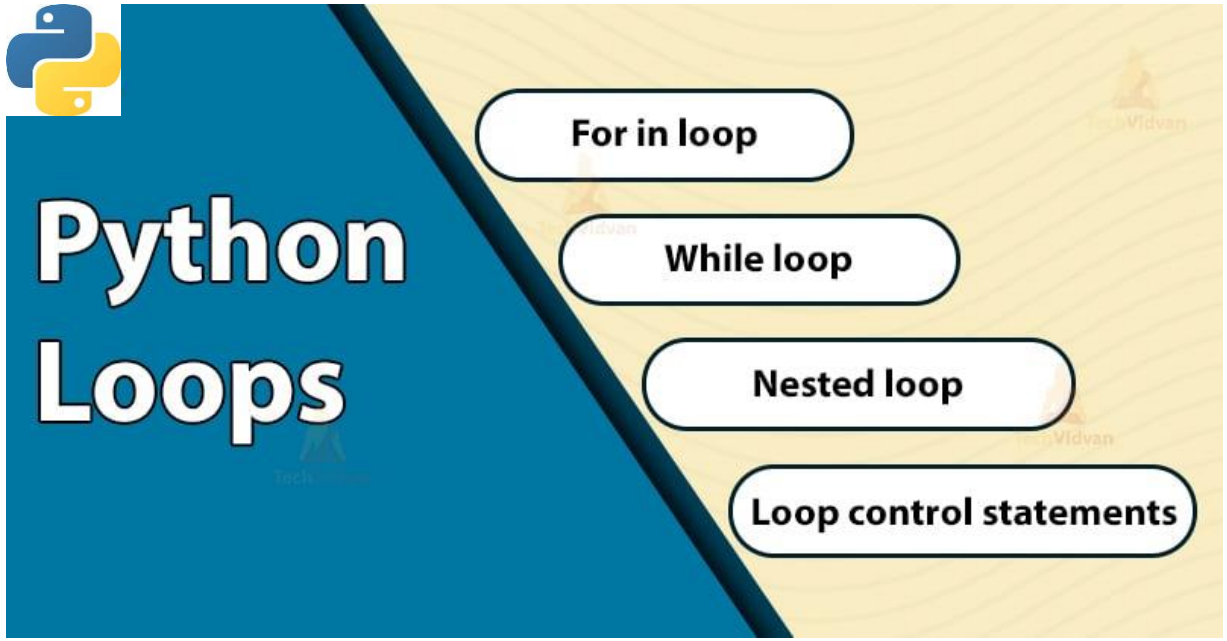
- Python Program Structure and Flowcharts
- Variables and Python Data Model (Data Types) – int, float, str and bool
- Type Checking and Casting, Basic Arithmetic, Strings, Input and Output,
- Program Blocks
  - Sequential Blocks
  - Conditional Structure (if-elif-else)
    - Boolean Operators, Truth tables, Comparison Operators, membership and identity operators
  - Iterations and Loops

COM161 (75777) - Problem Solving for Computing. 2025-26.

COM161 - Problem Solving for Computing

Lecture 4

Repetition Structures and Loops in Python



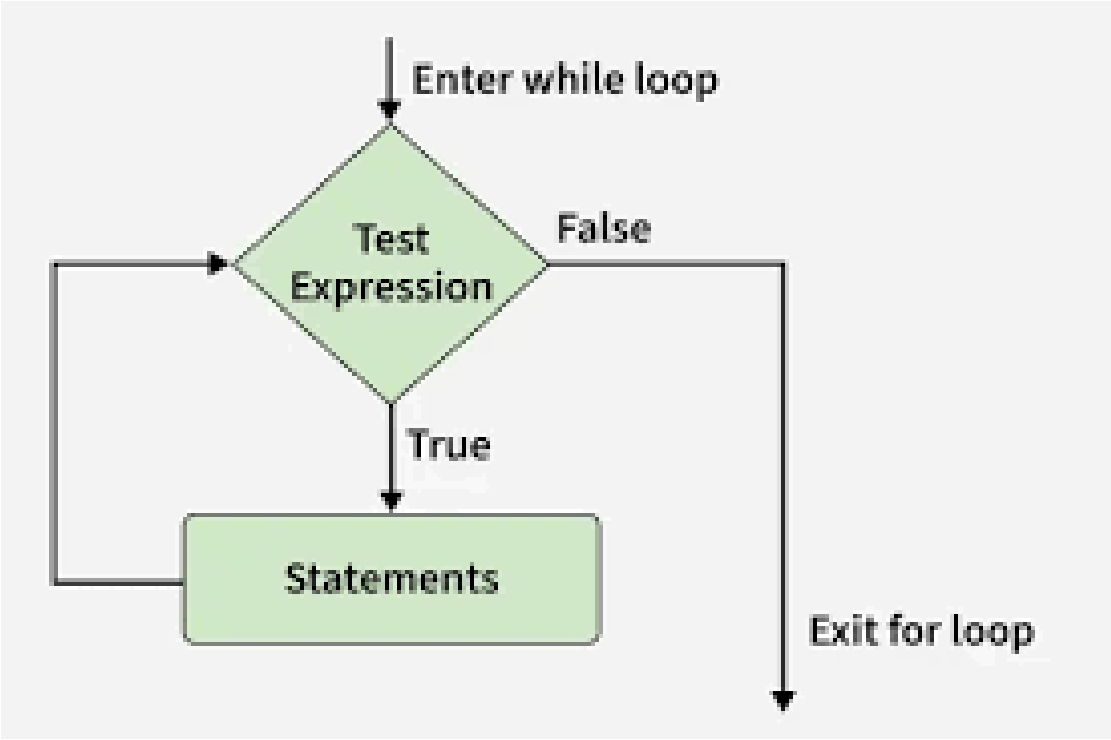
Mastery of Repetition Structures in Python

Learning Objectives

- Understand the purpose of loops.
- Differentiate between condition-controlled and count-controlled loops.
- Master while and for loop syntax.
- Implement input validation and sentinels.
- Apply loops to Turtle Graphics.

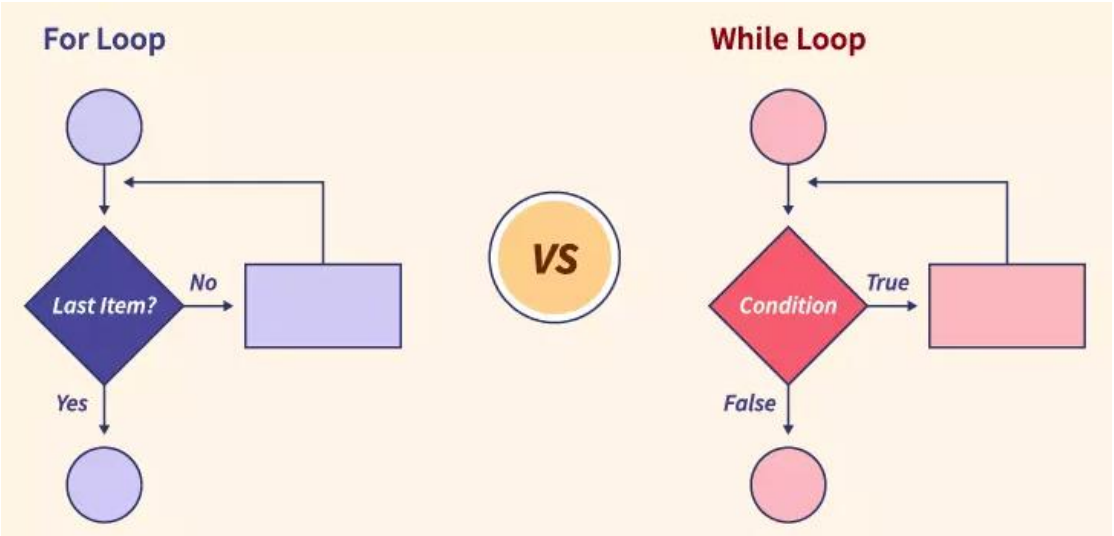
Why Do We Need Repetition?

- Programs often need to perform the same task multiple times.
- Duplicating code is inefficient, makes the script long, and is a nightmare for debugging or making changes.
- **Question:** "If you had to print 'Hello' 1,000 times, would you write 1,000 print statements?"



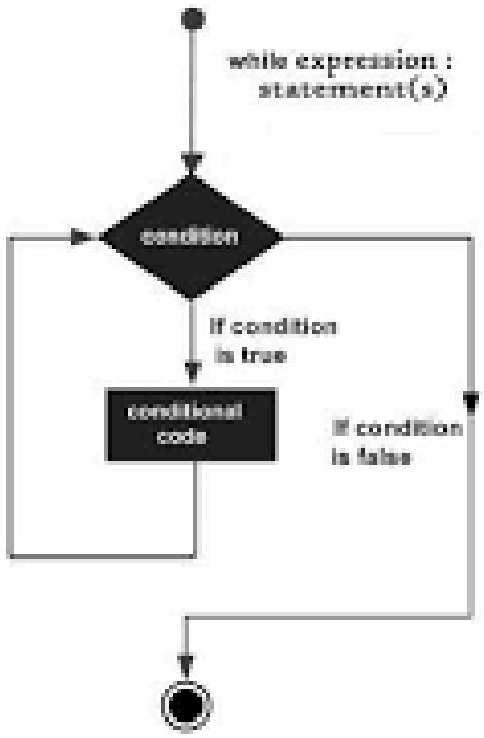
Definition of a Repetition Structure

- A repetition structure (or **loop**) causes a statement or set of statements to execute repeatedly.



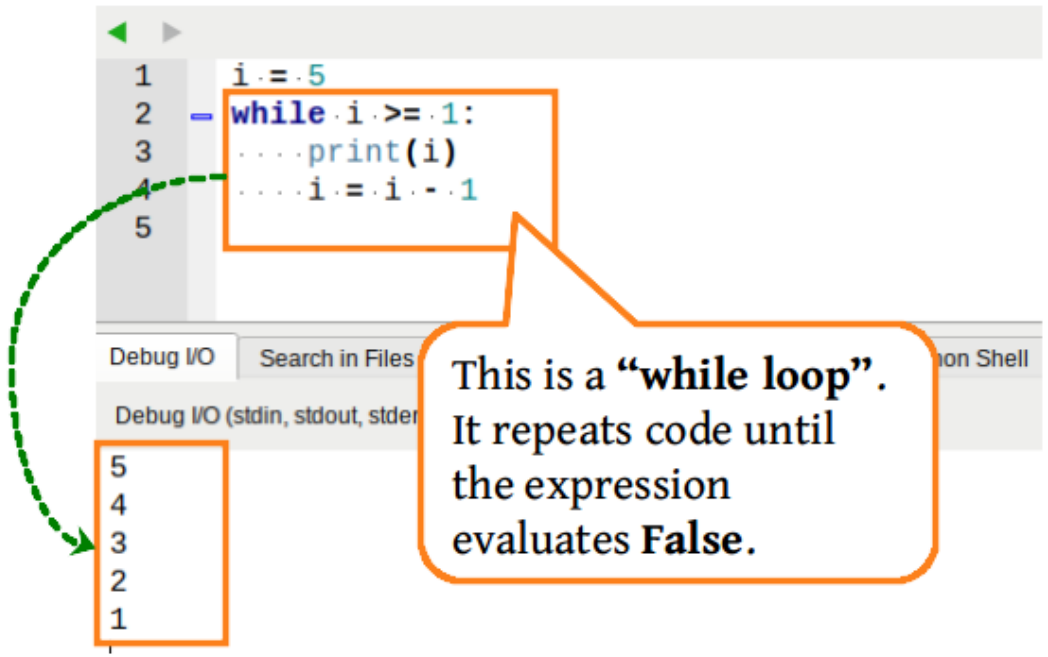
Two Broad Types of Loops

- **Condition-Controlled Loops:** Continue as long as a particular condition is True.
- **Count-Controlled Loops:** Iterate a specific number of times.



The `while` Loop

Operates while a specified condition is True.



Python Syntax for while

while condition:

statement

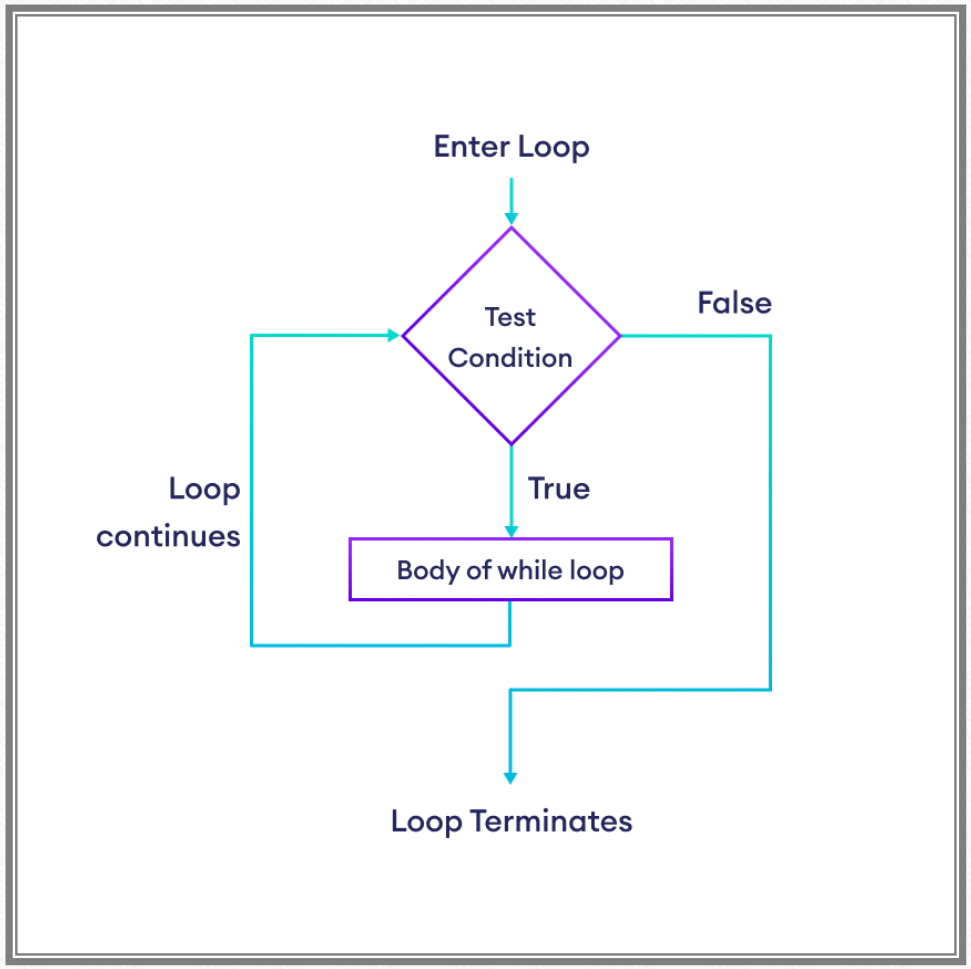
statement

- **Key Detail:** Don't forget the **colon** and the **indentation**.

How It Works?

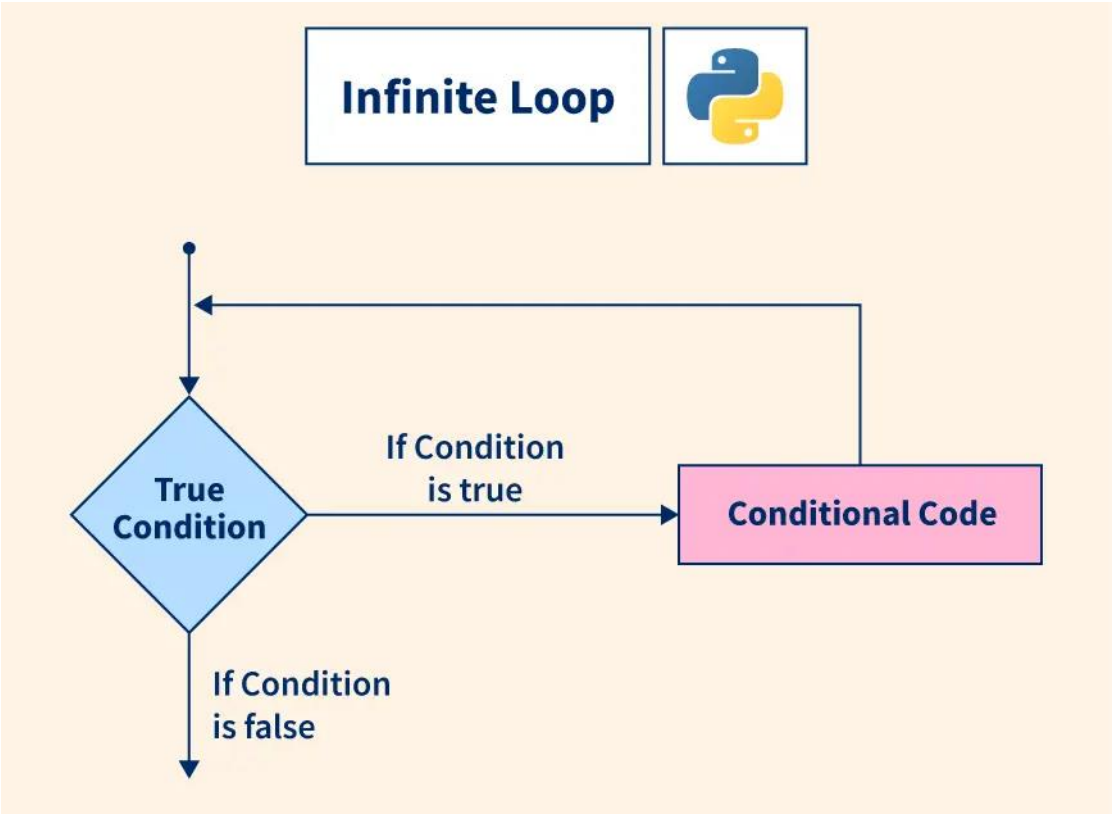
If the condition is True, the indented block executes, then the program loops back to the top.

If the condition is False, the block is skipped, and the program moves to the unindented code.



The Pretest Nature of while

- Known as a **pretest loop** because the condition is tested *before* the first iteration.
- **Key Detail:** To enter the loop the first time, the condition must be initialized to True earlier in the code.



Warning: The Infinite Loop

- If a statement inside the loop never makes the condition False, the loop will never stop.
- One statement *inside* the loop must change the control variable.

```python
# This program calculates sales commissions.
# Create a variable to control the loop.
keep_going = 'y'

# Calculate a series of commissions.
while keep_going == 'y':
    # Get a salesperson's sales and commission rate.
    sales = float(input('Enter the amount of sales: '))
    comm_rate = float(input('Enter the % commission rate: '))

    # Calculate the commission.
    commission = sales * comm_rate/100

    # Display the commission.
    print('The commission is £', \
            format(commission, '.2f'),sep='')

    # See if the user wants to do another one.
    keep_going = input('Do you want to calculate another ' + \
                       'commission (Enter y for yes): ')
# end of while loop
print('OK - goodbye.')
```

Example commission.py

- keep_going = 'y' # Initialization
- while keep_going == 'y':
- # (Processing happens here)
- **Explanation:** The variable **keep_going** is the control variable.

Example: commission.py

```
# Get a salesperson's sales and commission rate.
sales = float(input('Enter the amount of sales: '))
comm_rate = float(input('Enter the % commission rate: '))

# Calculate the commission.
commission = sales * comm_rate/100

# Display the commission.
print('The commission is £', \
        format(commission, '.2f'),sep='')
```

Example: commission.py (The Exit Strategy)

keep_going = input('Do you want to calculate another? (y for yes): ')

This line allows the control variable to change, preventing an infinite loop.

```
# See if the user wants to do another one.
keep_going = input('Do you want to calculate another ' + \
                   'commission (Enter y for yes): ')
```

Flowcharting the Commission Program

- Start
- Assign
- Condition
- Statements
- Re-assign loop

while Loop Practice Task

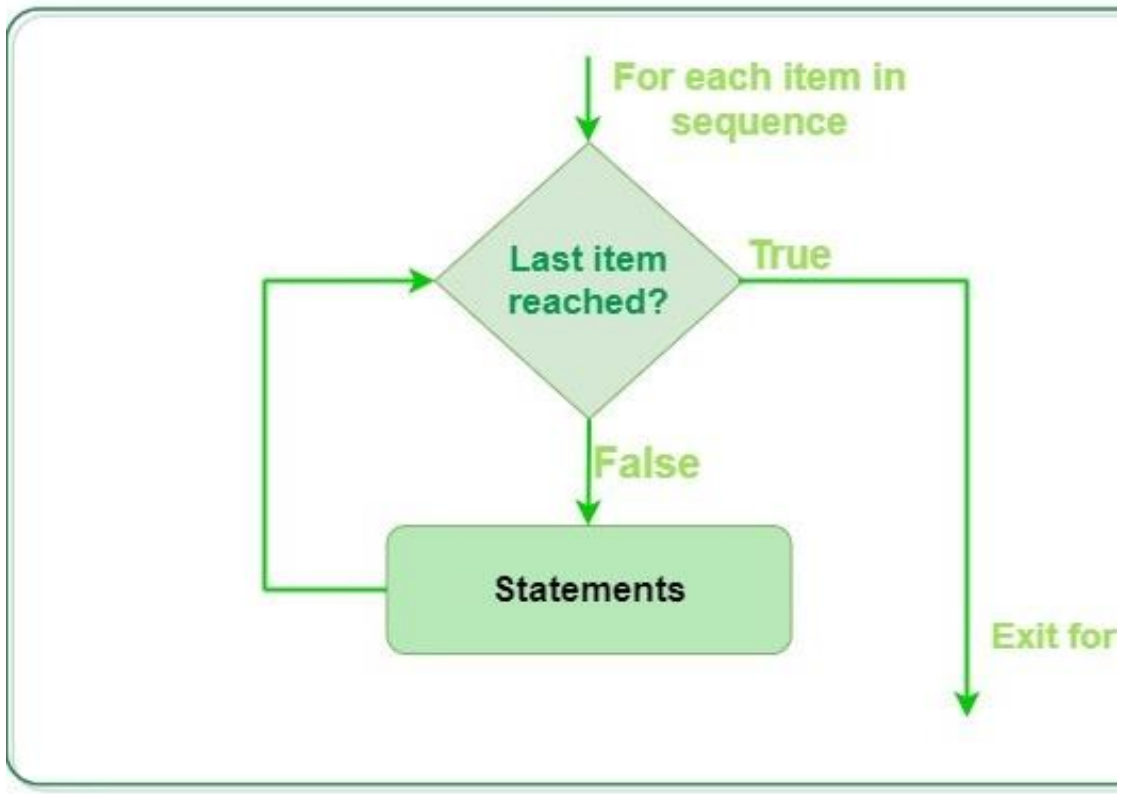
```python
correct_password = "secure123"

while True:
    user_input = input("Enter password: ")

    if user_input == correct_password:
        print("Access granted.")
        break
    else:
        print("Incorrect password. Try again.")
```

- Draft a simple while loop that asks for a password until they get it right.

COM161 (75777)- Problem Solving for Computing. 2025-26.

Part 3

The `for` Loop



The for Loop: Count-Controlled

- Iterates a specific number of times.
- **Key Detail:** The variable in a for loop is often called the **target variable**.

Using Lists

```
for variable in [value1, value2, ...]:
    statement
```

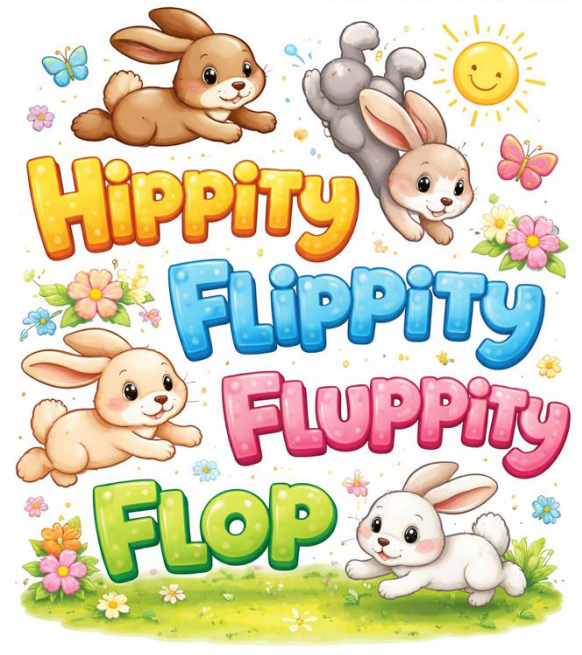
- **Logic:** The variable is assigned each value in the list sequentially

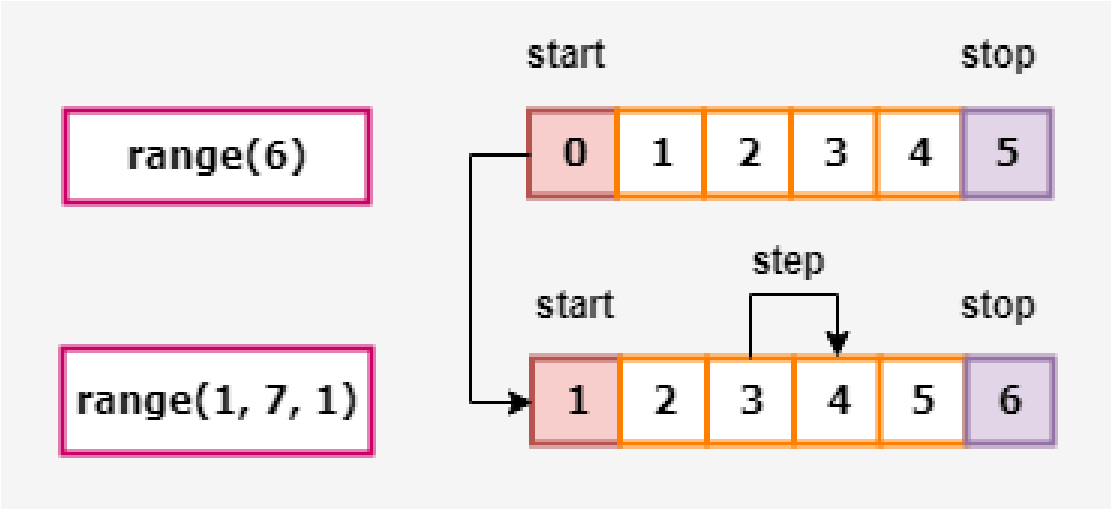
```
>>> for num in [1,3,5,7,9,11]:
...     print(num)
...
1
3
5
7
9
11
>>>
```

List Example: Numbers

List Example: Strings

```
>>> for name in ['Hippity', 'Flippity', 'Fluppity', 'Flop']:
...     print(name)
...
Hippity
Flippity
Fluppity
Flop
```





The range() Function

- A powerful way to write count-controlled loops.

range() with One Argument

Content: range(5)

Logic: Starts at 0, ends at 4 (one less than the specified number).

Remember: "Python counts starting from zero!"

range() with Two Arguments

Content: range(2, 6)

Logic: Starts at 2, ends at 5 (one less than the second number).

range() with Three Arguments

Content: range(1, 11, 2)

Logic: Start at 1, step size of 2, ends before 11 (1, 3, 5, 7, 9).

Example Table of Squares/Cubes

- **Content:** Using a loop to print a table from 1 to 20.
- **Code:**

```
for number in range(1, 21):
    print(number*number)
```

- Using the [**] operator (number**2)

```
base = 2
exponent = 3
result = base ** exponent
print(result)  # Output: 8
```

Stepping Backwards

- **Content:** Using a negative step value.
- **Code:** range(20, 0, -1)
- This allows stepping backwards from 20 to 1.

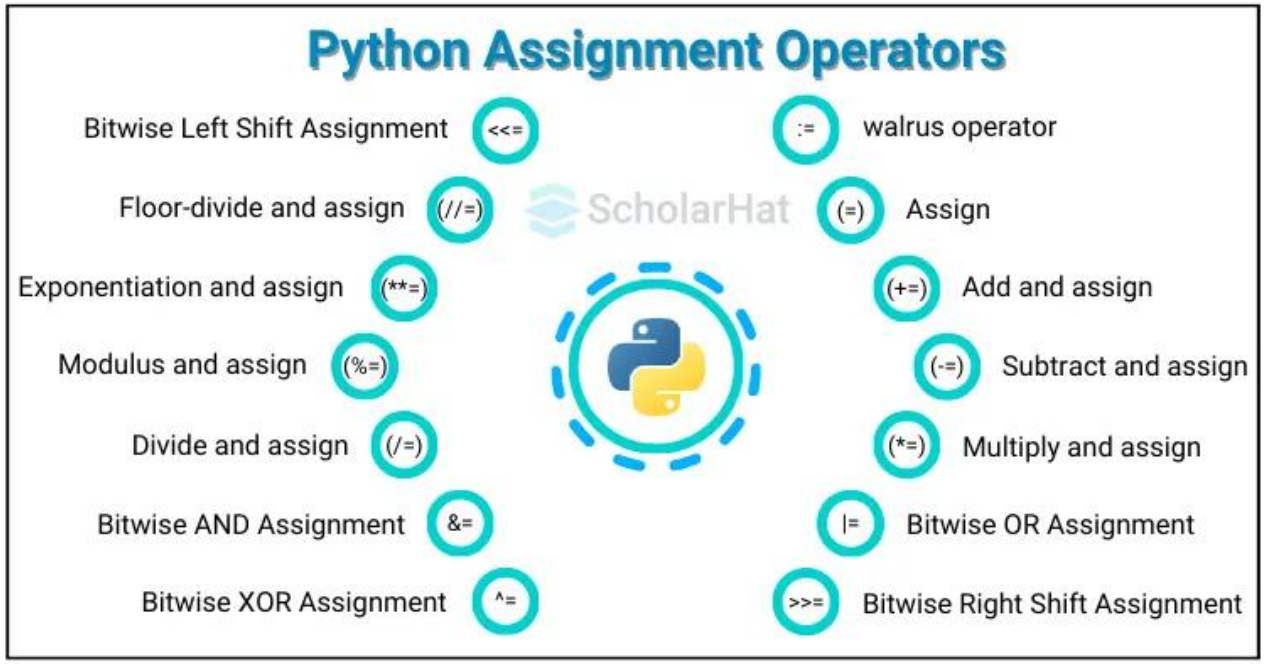
COM161 (75777)- Problem Solving for Computing. 2025-26.

Part 4

Augmented Assignment & Sentinels

Augmented Assignment Operators

- **Content:** Shorthand for updating variables.
- **Example:** x = x + 2 is equivalent to x += 2.



Augmented Assignment Operators, Examples

```
>>> b=12
>>> b-=1
>>> print(b)
11
>>> b=-2
>>> print(b)
-2
>>> b*=2
>>> print(b)
-4
>>> b-=2
>>> print(b)
-6
```

What is a Sentinel?

```
sum = 0
data = int(input("How long? (enter 0 to end) "))
while data != 0:
    sum = sum + data
    data = int(input("How long? (enter 0 to end) "))
print("Sum:",sum)
```

- **Content:** A special value that marks the end of a sequence of items.
- **Explanation:** When the program reaches the sentinel, the loop terminates.
- **The input value 0** is the sentinel value for these loops.

Sentinel Best Practices

- **Content:**
  - Make sure the user knows what the sentinel is.
  - The sentinel must not be mistaken for a regular value in the sequence.

COM161 (75777) - Problem Solving for Computing. 2025-26.

Part 5

Input Validation & Nested Loops

Input Validation: GIGO

- **Garbage In, Garbage Out" (GIGO)** is a foundational principle in computer science, data analytics, and artificial intelligence, asserting that the quality of output is determined by the quality of the input
- **Logic:** Checking data to ensure it is valid before using it in a calculation.

The Validation Loop Structure

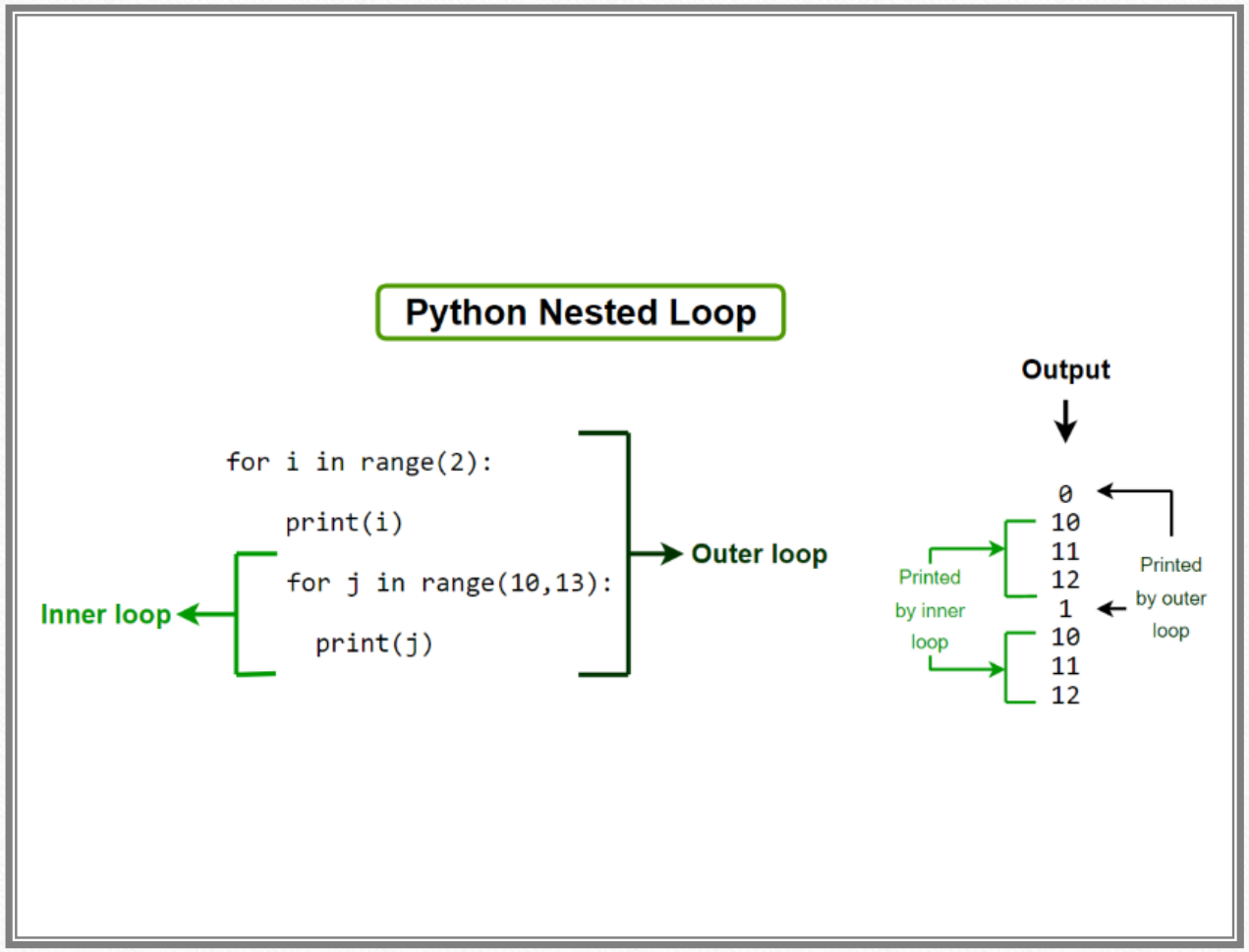
- **Key Detail:** Input is read in two places:
  - Before the loop (the **priming read**).
  - Inside the loop (to get a new value if the first was "bad").

```
while True:
    diffChoice = input("Select a difficuilty...\n[e]asy, [m]edium,
    [h]ard: ").lower()

    if diffChoice in ('e', 'm', 'h', 1, 2, 3):
        break
    else:
        diffChoice = input("Incorrect input...\nSelect a
        difficuilty...\n[e]asy, [m]edium, [h]ard: ").lower()
```

Example: Retail Validation

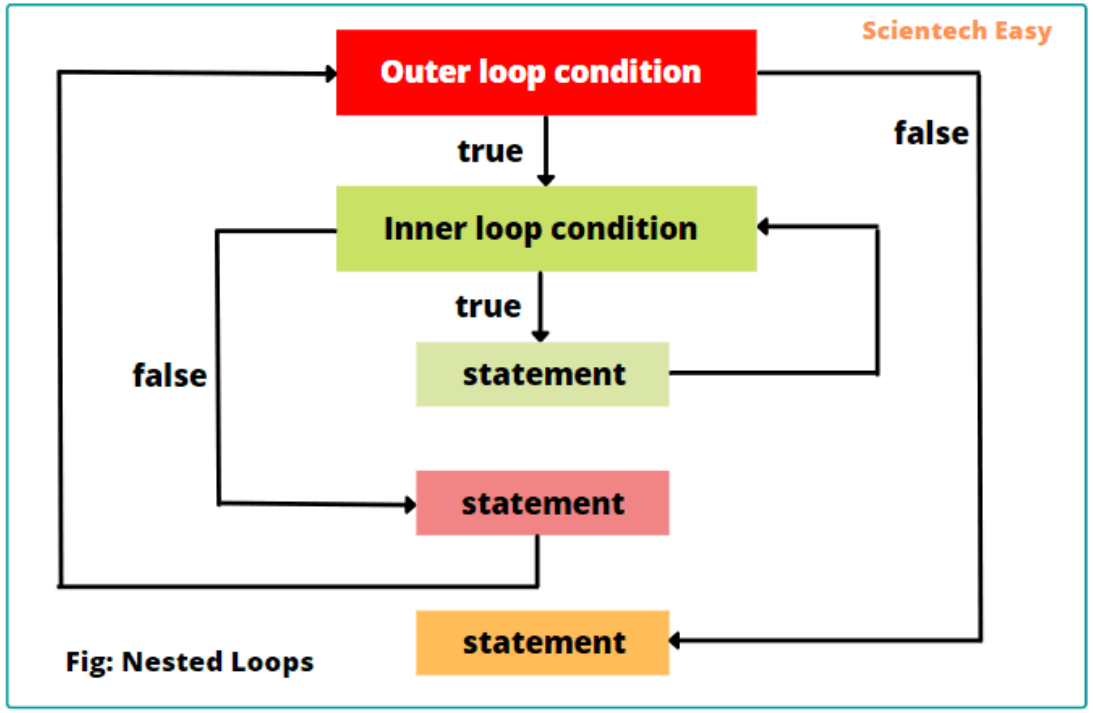
- **Content:** Validating that wholesale cost is not negative.
- **Code:**

```
wholesale = float(input('Enter cost value: '))
while wholesale < 0:
    print('ERROR: cannot be negative.')
    wholesale = float(input('Enter correct cost: '))
```



Nested Loops

- **Content:** A loop inside another loop.
- **Example:** A validation loop inside a program control loop.



Visualizing Nested Loops

- **Explanation:** For every single iteration of the *outer* loop, the *inner* loop runs to completion.

COM161 (75777) - Problem Solving for Computing. 2025-26.

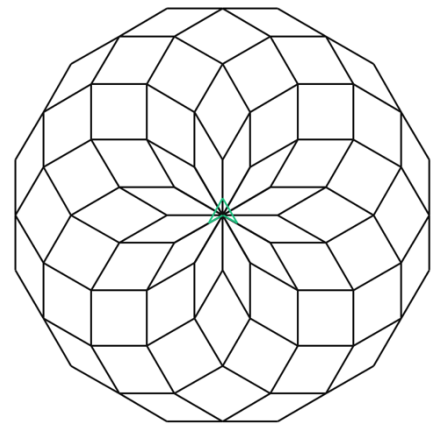
Part 6

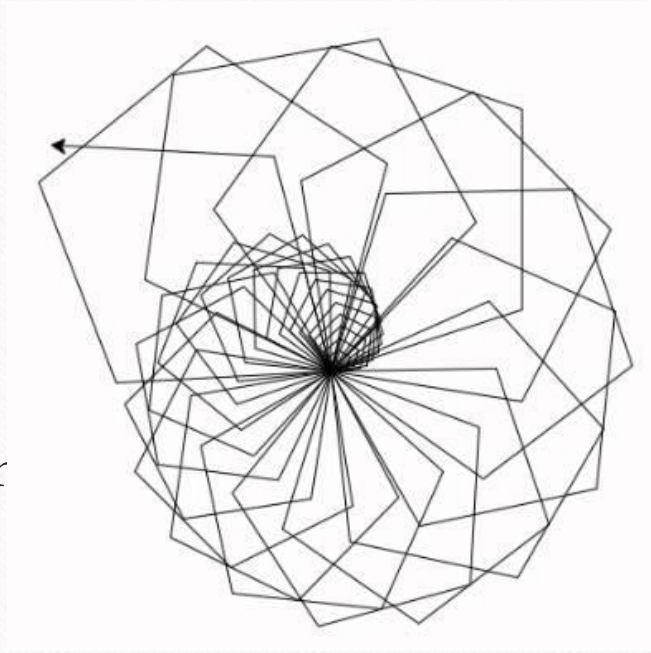
Loops in Graphics & Conclusion

Efficiency in Turtle Graphics

- **Content:** Instead of writing turtle.forward(100) and turtle.left(90) four times, use a loop.

```
for i in range(4):
    turtle.forward(100)
    turtle.left(90)
```





- By nesting loops in Turtle, we can create complex geom
code.

Summary & Key Takeaways

- while for unknown number of iterations.
- for for known ranges or lists.
- Always ensure your loop has an exit condition to avoid infinite loops.
- Use input validation to protect your program (GIGO).

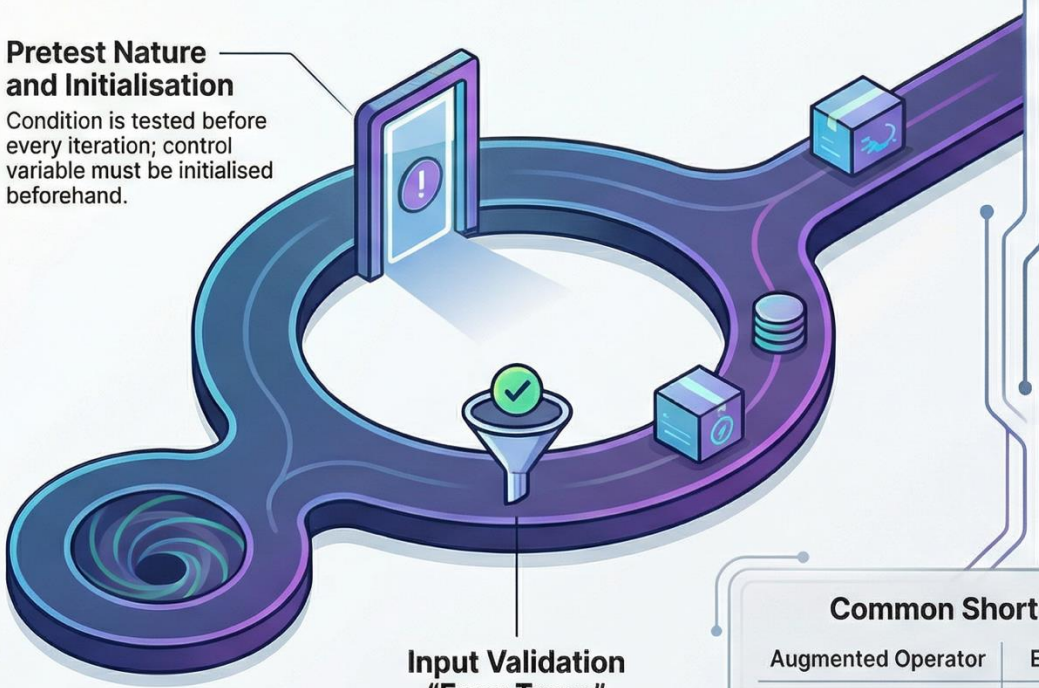
Mastering Python Repetition Structures: while vs for

Repetition structures (loops) execute statements multiple times. This compares condition-controlled `while` loops with count-controlled `for` loops, highlighting syntax and safety.

`while` Loop: Condition-Controlled

Pretest Nature and Initialisation

Condition is tested before every iteration; control variable must be initialised beforehand.



Infinite Loop Risk

At least one statement must eventually make the condition false.

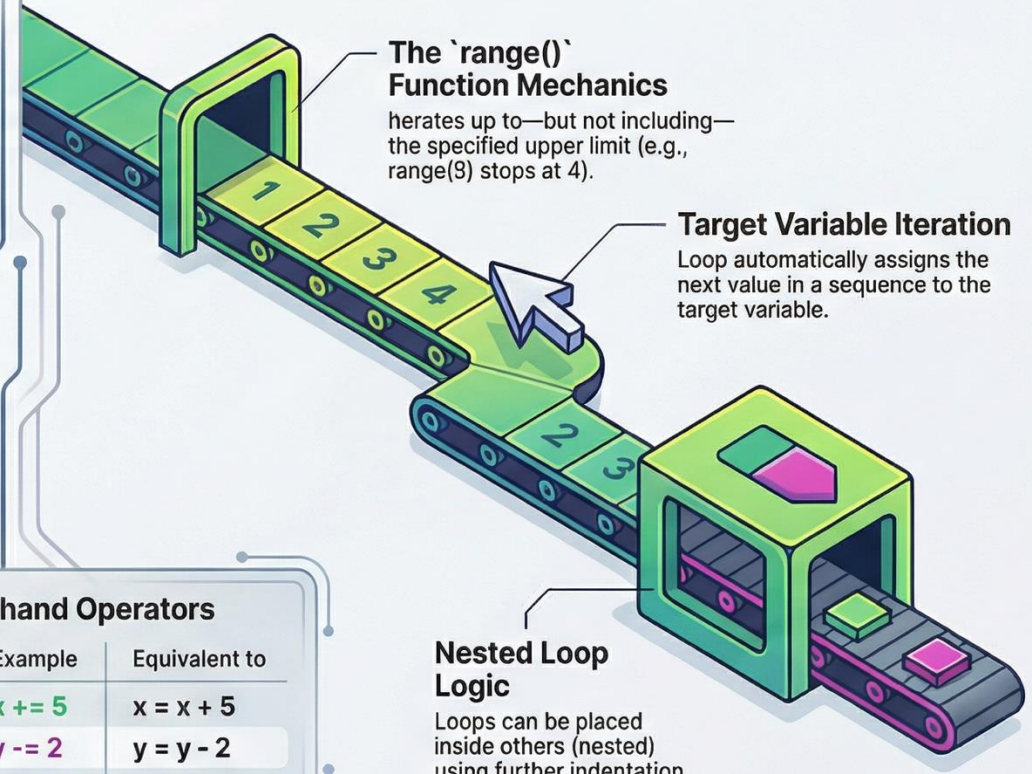
Input Validation "Error Traps"

Loops can force a 'priming read' to check data validity before processing it.

`for` Loop: Count-Controlled

The `range()` Function Mechanics

herates up to—but not including—the specified upper limit (e.g., range(3) stops at 4).



Target Variable Iteration

Loop automatically assigns the next value in a sequence to the target variable.

Nested Loop Logic

Loops can be placed inside others (nested) using further indentation for complex tasks.

Common Shorthand Operators

Augmented Operator	Example	Equivalent to
+=	x += 5	x = x + 5
-=	y -= 2	y = y - 2
*=	z *= 6	z = z * 6

NotebookLM

Q&A & Lab Introduction

- Floor is open for questions.
- Upcoming lab you will write some programs to implement the loop concepts and techniques.